Fundamentals of Programming using Python • First Year, FinTech

TP 03 — Practical Work

Variables and the Basic Commands

8 exercises • no answers in this sheet

Dr. Brahim Zirari

Yahia Farès University of Medea

in linkedin.com/in/brahimzirari · Google Scholar

October 2026

How to work through this TP

- Do the exercises **in order**. Each one uses only what Chapter 03 and the chapters before it have taught — nothing else is needed, and nothing else is expected.
- **Type the code yourself.** Do not copy and paste. Typing is what builds the memory, and you cannot fix an error you did not write.
- Run your file and compare your output with what the exercise asks for. If they differ, read the error message from the bottom up: the last line names the real mistake.
- Only then open the companion sheet *Solutions — Chapter 03*. Reading the answer before trying teaches you nothing.

❑ **What to hand in** – One Python file per exercise that produces code, named `ex1.py` to `ex8.py`, plus a short written answer for the exercises that ask you to explain rather than to compute.

The 8 exercises

Exercise 1. Legal or not (written answer) – For each name, say whether Python accepts it, and if not why: (a) `total_due`; (b) `2nd_client`; (c) `Total`; (d) `class`; (e) `taux_intérêt`; (f) `unit price`; (g) `_tmp`; (h) `sum`. For (h), say what Python does and why it is still a bad idea.

Exercise 2. Read it right-to-left (written answer) – Without running anything, give the value printed by each of these three programs, and explain the third in one sentence. (a) `x = 5` then `x = 8` then `print(x)`. (b) `x = 5` then `y = x` then `x = 8` then `print(y)`. (c) `x = 5` then `x = x + x` then `x = x + x` then `print(x)`.

Exercise 3. Swap and trace (code) – Write `ex3.py`: a cashier's drawer holds 48 000 DZD and the safe holds 350 000 DZD. Store both, move 25 000 DZD from the safe to the drawer using augmented assignment, then exchange the two amounts in a single line, and print them. Before running it, write the trace table by hand.

Hint: `a, b = b, a` swaps; `+=` and `-=` move amounts.

Exercise 4. A running total (code) – A current account starts at 0 DZD. Apply, in order: a deposit of 120 000, a card fee of 450, a transfer out of 38 500, and interest-free credit of 15 000. Use augmented assignment for every movement and print the balance after the last one, to two decimals. Then say what happens if the first line is deleted.

Hint: start the total at `0` on a line of its own.

Exercise 5. Formatting a receipt (code) – Using `sep`, `end` and `\t`, produce exactly these three lines, with the columns lined up by tabs: `Item\tQty\tDZD`, then `Card\t2\t900`, then `Transfer\t1\t450`. Print the date line `2026-10-15` using `sep`, from the three separate values `"2026"`, `"10"` and `"15"`.

Hint: `sep` goes between the values, `end` after the last.

Exercise 6. One line, two prints (code) – Produce the single output line `Processing... done (3 files)` using **three** separate `print` calls. Explain in one sentence which option makes this possible.

Hint: what does `print` put after the last value by default?

Exercise 7. Read, convert, compute, show (code) – Write `ex7.py` following the five-step shape, one job per line. It asks for a client name, a number of units and a unit price in DZD; it computes the amount before tax, the VAT at 19%, and the total; it prints three lines, each amount to two decimals. Since a program that asks cannot be marked automatically, also write at the top, as a comment, the three values you tested with and the three lines they produced.

Hint: `input()` returns text — convert before you calculate.

Exercise 8. Find the four faults (written answer) – This program is meant to print the balance after a fee. It has four separate faults. Name each one, say what Python does about it (error, or silently wrong), and give the corrected program.

```
1 print(balance)
2 balance = 50000
3 2nd_fee = 1250
4 sum = balance - 2nd_fee
5 Balance = sum
6 print("Left:", balance)
```